

RSCA_NoDB Project README

This project is designed to perform a Regional Stream Condition Assessment (RSCA) without requiring a direct, constant connection to a database. It processes stream data, compares it to reference sites, and generates reports and visualizations to assess stream health.

Project Structure

- **Main.R:** The main script that orchestrates the entire workflow. It reads a configuration file, loads data, and runs the analysis steps in sequence.
- **config.R:** A configuration file used to set parameters for the analysis, such as input file paths, output directories, and processing options. This allows for easy modification of the analysis without changing the R code.
- **R/:** This directory contains all the R scripts that perform the actual analysis.
 - **0.1_Data_Prepping.R:** This script is responsible for connecting to the database, fetching the necessary data, and saving it to an RData file (**Base_Files/Base_Data.RData**). This script only needs to be run when the underlying database is updated (e.g., bi-annually).
 - **0.2_RSCA_Core.R:** The core of the analysis. It takes the prepared data and a list of test sites and performs the RSCA analysis.
 - Other numbered scripts (**1.0_...**, **2.0_...**, etc.): These scripts contain the functions for specific analysis steps, such as CSCI checks, comparator site selection, and Line of Evidence (LOE) analysis.
 - **RSCA_Graphing.R** and **plots.R:** Scripts for generating plots of the results.
- **input/:** This directory should contain the CSV files with the list of "test sites" to be analyzed. The path to the specific file to use is set in **config.R**.
- **output/:** This directory is where all the output files (CSVs, Excel reports, plots) are saved. The base output directory is configured in **config.R**.
- **Base_Files/:** This directory contains the base data required for the analysis.
 - **Base_Data.RData:** An RData file containing all the necessary data fetched from the database by the **0.1_Data_Prepping.R** script. This file is loaded by **Main.R** to perform the analysis, avoiding the need for a database connection during the main workflow.

Contents of Base_Data.RData:

Object Name	Source Database Table(s)	Description
csci_base_df	sde.analysis_csci_core	California Stream Condition Index scores, percentiles, and metrics for all sites and samples
stressor_csci_base_df	sde.lab_chem, sde.field_chem, analysis_phabmetrics	All stressor and chemistry data (lab chemistry, field chemistry, and PHAB metrics) merged and unified for all sites

Object Name	Source Database Table(s)	Description
oe_base_df	sde.analysis_csci_suppl1_oe	Expected taxa and associated capture probabilities (O/E data) for all samples
station_base_df	sde.lu_stations + sde.analysis_csci_suppl1_oe	Station location and metadata (coordinates, county, HUC, COMID, station name)
chansum_df	sde.unified_channelengineering_summary	Channel engineering classifications for all sites
scape_base_df	sde.scape_strm_constraints	Stream constraints and SCAPE data for reference condition assessment (moved from R/0.2_RSCA_Core.R to eliminate database calls during analysis)

- **Note:** To regenerate this file when the source database has been updated, run the **0.1_Data_Prepping.R** script with a valid database connection.

- **python/**: Contains auxiliary Python scripts.

Workflow

1. **Configuration (Required):** Before running anything, edit **config.R** to set up your analysis parameters. This is the ONLY file you need to edit. The configuration file controls:

- **run_data_prep:** Set to TRUE if you need to regenerate the base data from the database (only needed when database is updated). Default: FALSE.
- **run_analysis:** Set to TRUE to run the main RSCA analysis. Default: TRUE.
- **Type:** Channel Engineering Type for filtering sites (or NA to use values from input CSV).
- **graph_mode:** Controls graph generation ("none", "primary", "secondary", "both").
- **merge_csvs:** Whether to merge output CSV files (TRUE/FALSE).
- **chunk_start:** Starting chunk for processing (useful for resuming interrupted runs).
- **output_base_dir:** Directory where results will be saved.
- **import_sites_path:** Path to the CSV file containing the list of sites to analyze.

2. **Data Preparation (if necessary):** If the data in the database has been updated:

- Set **run_data_prep <- TRUE** in **config.R**.
- Make sure you have a valid database connection (con).

- Run **Main.R**. It will execute **R/0.1_Data_Prepping.R**, which fetches the data and saves it to **Base_Files/Base_Data.RData**.
- The analysis will then run automatically (if **run_analysis <- TRUE**).
- After the update is complete, set **run_data_prep <- FALSE** to avoid re-running unnecessary database queries.

3. **Running the Analysis:** Once **config.R** is set up with **run_analysis <- TRUE**, simply execute **Main.R**. It will:

- Load the configuration from **config.R**.
- (Optional) Regenerate base data if **run_data_prep <- TRUE**.
- Load the base data from **Base_Files/Base_Data.RData**.
- Load the list of test sites from the CSV file specified in the configuration.
- Process the sites in chunks. For each chunk, it runs the core RSCA analysis (**R/0.2_RSCA_Core.R**).
- Generate output files (Excel reports, CSVs, and plots) for each site and save them in the configured output directory.
- If **merge_csvs** is set to TRUE, it will merge the individual CSV files into summary files at the end of the process.

Data Path and Processing

- **Input Data:** The analysis starts with a list of sites provided in a CSV file (e.g., **input/Hot_Creek_Sites.csv**).
- **Base Data:** The **Main.R** script loads **Base_Files/Base_Data.RData**, which contains all the necessary data for CSCI scores, stressors, and other metrics. This data is pre-processed from the database.
- **Processing:**
 - The sites from the input file are validated against the CSCI data in the base data file.
 - The valid sites are then processed in chunks to manage memory usage.
 - For each site, the **0.2_RSCA_Core.R** script is executed, which in turn calls various functions from the other **R/** scripts to perform the analysis.
 - The analysis involves several steps, including comparator site selection, calculating different Lines of Evidence (LOE), and summarizing the results.
- **Output:** The results are written to the directory specified by **output_base_dir**. For each site, a subdirectory is created, containing:
 - An Excel file with summary data (**<site_id> Summary_Site_Data.xlsx**).
 - An Excel file with monitoring recommendations (**<site_id> Monitoring_Recommendations.xlsx**).
 - Several CSV files with detailed data for each analysis step.
 - Plots, if **graph_mode** is enabled.
- **Merged Output:** If **merge_csvs** is true, the script will also create merged CSV files in the **output_base_dir** that combine the data from all processed sites.

R Script Details

- **R/0.1_Data_Prepping.R:** Connects to the SMC database to pull and clean all necessary data, including station information, CSCI scores, chemistry data, and physical habitat metrics. It assembles this data into several base dataframes and saves them into a single RData file: **Base_Files/Base_Data.RData**. This script is meant to be run infrequently, only when the source database is updated.
- **R/0.2_RSCA_Core.R:** This is the central script that orchestrates the RSCA analysis for a given list of test sites. It sources all the individual analysis function scripts (1.0 through 6.0). It then iterates through each test site, running the full sequence of RSCA functions: CSCI check, comparator selection, data prep for each

Line of Evidence (LOE), running each LOE analysis, and summarizing the results. Finally, it extracts and combines the results from all sites into several summary dataframes.

- **R/1.0_Test_CSCI_check.R:** Contains the **CSCI_check_fun** function. This function checks if a given test site has a CSCI score and compares it against a threshold (either a default value of 0.79 or a dynamic value from SCAPE data). This determines if the site is "passing" and whether the full causal assessment is needed.
- **R/2.0_Comparator_Site_Selection_v2.R:** Contains the **Comp_Select_Modified_fun** function. This function identifies suitable comparator sites for a given test site from the base data. It uses Bray-Curtis dissimilarity based on O/E (Observed/Expected) taxa data to find biologically similar sites, filtering them by a maximum dissimilarity threshold. It can also filter sites based on their channel engineering class.
- **R/3.0_LOE_Data_Prep.R:** Contains functions (**SCO_dat_fun**, **RCC_dat_fun**, **SR_log_dat_fun**) to prepare the data for each of the three Lines of Evidence (LOE): Spatial Co-Occurrence, Reference Condition Comparison, and Stressor-Response. These functions take the test site and its selected comparators and create tailored dataframes for each specific analysis.
- **R/4.1_Spatial_CoOccurrence_LOE.R:** Contains the **SCO_fun** and **SCO_sum_mod** functions. This LOE compares the test site's stressor levels to the distribution of stressor levels at biologically similar comparator sites where the CSCI score is higher than the test site.
- **R/4.2_Ref_Condition_Comp_LOE.R:** Contains the **RCC_fun** and **RCC_sum_mod** functions. This LOE compares the test site's stressor levels to the distribution of stressor levels at reference-condition comparator sites (i.e., sites with a CSCI score ≥ 0.79).
- **R/4.3_Stressor_Response_LOE.R:** Contains the **SR_log_fun** and **SR_log_mod_sum** functions. This LOE uses logistic regression models built from the comparator sites to predict the probability of a poor CSCI score at the test site based on its stressor levels.
- **R/5.0_LOE_summarize.R:** Contains the **LOE_sum_fun** and **LOE_samp_sum_fun** functions. These functions aggregate the scores from the individual LOE analyses to produce an overall assessment for each stressor module (e.g., Conductivity, Eutrophication) for each test site sample.
- **R/6.0_Data_Inventory.R:** Contains the **Dat_invnt_fun** function. This function inventories the available stressor data for both the test sites and their comparators, identifying data gaps and assigning a priority for future monitoring.
- **R/RSCA_Graphing.R:** This script (**site_csci_module_plotter**, **site_loe_plotter**) generates the "primary" summary plots for each test site, including a time-series of CSCI scores and a tile plot showing the overall module assessment results over time. These are saved as JPEG files in the site's output directory.
- **R/plots.R:** This script contains the functions (**spatial.co.plot**, **ref.cond.plot**, **stress.resp.plot**) that generate the "secondary" detailed plots for each Line of Evidence. These plots visualize the underlying data for each assessment (e.g., boxplots of comparator data, logistic regression curves).
- **R/plumber.R:** This script (**last_plots**) serves as a wrapper to execute the secondary plotting functions from **plots.R**. It iterates through all the processed sites, samples, and modules to generate and save all the detailed LOE plots as PNG files in the **Secondary_graphs** subfolder of each site's output directory.